

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import cvxpy as cp
import yfinance as yf
```

Data Processing

```
# Define tickers
tickers = ["XLB", "XLE", "XLF", "XLI", "XLK", "XLP", "XLU", "XLV", "XLY"] #weekly data, weekly avg, weekly std, trade weekly

# 3 months of weekly data, out of the sample,

# Fetch historical data from August 1999 onwards
start_date = "2020-01-01"
end_date = "2020-12-31"

# Download data
etf_data = yf.download(tickers, start=start_date, end=end_date, group_by='ticker')
```

```
print(etf_data)
```

```
YF.download() has changed argument auto_adjust default to True
[*****100%*****] 9 of 9 completed
Ticker
Price      Open      High      Low      Close      Volume      Open
Date
2020-01-02  55.595272  55.694177  54.516275  54.579216  7357400  54.803554
2020-01-03  54.021739  54.345435  53.680056  53.698040  12423200  54.066971
2020-01-06  53.545170  53.796938  53.419287  53.464245  15764400  54.066967
2020-01-07  53.374340  53.769971  53.230474  53.401314  20266900  54.050037
2020-01-08  53.410300  53.805931  53.230467  53.590130  8079600  54.134709
...
2020-12-23  65.634926  66.057665  65.331655  65.368416  4109600  53.665421
2020-12-24  65.423543  65.781952  65.166224  65.781952  1366300  53.419856
2020-12-28  66.397697  66.480404  65.497075  65.524651  2661300  53.963612
2020-12-29  65.873868  66.020904  65.037576  65.368416  3426400  54.051309
2020-12-30  65.588970  66.406879  65.552210  66.269028  2827500  53.946074

Ticker
Price      High      Low      Close      Volume      ...      XLK \
Date
2020-01-02  54.820484  53.821438  54.024635  19107700  ...      88.245570
2020-01-03  54.329430  53.939972  54.134701  17989300  ...      87.797202
2020-01-06  54.346363  53.982300  54.185497  10444500  ...      87.215244
2020-01-07  54.134701  53.804509  54.109303  13070300  ...      88.455439
2020-01-08  54.320976  53.999245  54.083912  12741400  ...      88.360042
...
2020-12-23  54.086395  53.297071  53.305840  5771700  ...      124.943108
2020-12-24  53.709277  53.130438  53.682964  3636200  ...      124.064027
2020-12-28  54.270575  53.718045  53.919762  5869500  ...      125.909104
2020-12-29  54.305644  53.761887  53.910984  7582500  ...      126.768859
2020-12-30  54.358279  53.858373  54.182873  5923600  ...      125.918808

Ticker
Price      High      Low      Close      Volume      ...      XLI \
Date
2020-01-02  89.094635  88.073849  89.094635  13283500  75.282534
2020-01-03  88.703512  87.787660  88.092941  15011800  75.291706
2020-01-06  88.398217  87.053065  88.302811  7815000  75.658726
2020-01-07  88.703482  88.131073  88.264633  7681800  75.759625
2020-01-08  89.600245  88.197855  89.209106  11627200  75.943126
...
2020-12-23  125.030045  123.667989  123.851532  5650400  82.368260
2020-12-24  124.981744  123.977090  124.672615  2192300  82.499327
2020-12-28  126.334149  125.319840  126.082993  4085500  82.929975
2020-12-29  126.933078  125.261893  125.484070  4420700  82.911246
2020-12-30  126.179619  125.232938  125.416481  4070700  82.134191

Ticker
Price      High      Low      Close      Volume
Date
2020-01-02  76.163353  75.080679  76.163353  16121300
2020-01-03  76.108299  75.227480  76.016548  17571300
2020-01-06  76.053260  75.484395  76.044083  16153100
2020-01-07  76.172506  75.502720  75.888077  16675400
2020-01-08  76.484466  75.814674  76.144981  10677700
...
2020-12-23  82.817639  82.265276  82.377617  8939400
2020-12-24  82.518048  81.918870  82.386978  2367400
```

[

]

$$r_{i,t} = \frac{P_{i,t} - P_{i,t-1}}{P_{i,t-1}} = \frac{P_{i,t}}{P_{i,t-1}} - 1$$

```
# 1. Calculate Arithmetic Returns
# Extract only 'Close' prices for all tickers
close_prices = etf_data.xs('Close', level=1, axis=1)

# Compute daily arithmetic returns
daily_returns = close_prices.pct_change().dropna()
print("Daily Arithmetic Returns:")
print(daily_returns)

# Extract Volume
volume_data = etf_data.xs('Volume', level=1, axis=1)

# 2. Calculate the Mean and the Covariance Matrix of the Percent Returns
mean_returns = daily_returns.mean()
print("\nMean returns:")
print(mean_returns)
covariance_matrix = daily_returns.cov().to_numpy()
print("\nCovariance Matrix:")
print(covariance_matrix)
```

```
2020-01-07 -0.001177 -0.001406 -0.001585 -0.001963 -0.002629 -0.006508
2020-01-08  0.003536 -0.000469  0.003016  0.006492 -0.016472  0.006551
2020-01-09  0.003692  0.005479  0.005302  0.004300  0.007369  0.006183
...
2020-12-23  0.001408 -0.001970  0.002162  0.000720  0.022139  0.016526
2020-12-24  0.006326  0.007075  0.002095  0.002879 -0.005219  0.001384
2020-12-28 -0.003911  0.004411  0.011401  0.002512 -0.006296  0.004836
2020-12-29 -0.002384 -0.000163  0.000313  0.004743 -0.006600 -0.003438
2020-12-30  0.013777  0.005043  0.006011 -0.000178  0.015679  0.004484
```

Ticker	XLP	XLK	XLI
Date			
2020-01-03	-0.001601	-0.011243	-0.001927
2020-01-06	0.002084	0.002382	0.000362
2020-01-07	-0.007679	-0.000432	-0.002052
2020-01-08	0.003708	0.010700	0.003385
2020-01-09	0.007067	0.011336	0.004218
...	...	...	...
2020-12-23	0.003325	-0.007586	0.005255
2020-12-24	0.004970	0.006630	0.000114
2020-12-28	0.007944	0.011313	0.001250
2020-12-29	-0.002974	-0.004750	-0.006015
2020-12-30	-0.001044	-0.000539	0.006394

[251 rows x 9 columns]

Mean returns:

Ticker

XLB	0.001068
XLU	0.000306
XLY	0.001215
XLV	0.000630
XLE	-0.000840
XLF	0.000253
XLP	0.000542
XLK	0.001686
XLI	0.000632

dtype: float64

Covariance Matrix:

```
[[0.00058687 0.0004664 0.00045767 0.00038615 0.00075409 0.00063225
  0.00034194 0.00049922 0.00056859]
 [0.0004664 0.00059203 0.00038821 0.00038065 0.00054644 0.00053289
  0.00037227 0.00044611 0.00048283]
 [0.00045767 0.00038821 0.00045424 0.00034268 0.00059684 0.00052348
  0.00029862 0.00049156 0.00047405]
 [0.00038615 0.00038065 0.00034268 0.00037069 0.00047988 0.00043777
  0.00029516 0.00042306 0.00039738]
 [0.00075409 0.00054644 0.00059684 0.00047988 0.00145836 0.00093885
  0.00040017 0.00060792 0.0008011 ]
 [0.00063225 0.00053289 0.00052348 0.00043777 0.00093885 0.00081951
  0.00039191 0.0005621 0.00067254]
 [0.00034194 0.00037227 0.00029862 0.00029516 0.00040017 0.00039191
  0.00030945 0.00036653 0.00035464]
 [0.00049922 0.00044611 0.00049156 0.00042306 0.00060792 0.0005621
  0.00036653 0.00064415 0.00050239]
 [0.00056859 0.00048283 0.00047405 0.00039738 0.0008011 0.00067254
  0.00035464 0.00050239 0.00062469]]
```

Apply Traditional Markowitz Model

```

# Portfolio Optimization Setup
mu0 = 0 # Cash holding return (risk-free rate)
n = len(tickers) # Number of assets

# Decision variables
x = cp.Variable(n) # Portfolio weights
x0 = cp.Variable(1) # Cash allocation

# Target return
R = 0.0008

# Define return and risk expressions
ret = mean_returns.to_numpy() @ x + mu0 * x0 # Portfolio return
risk = cp.quad_form(x, covariance_matrix) # Portfolio risk (quadratic form)

# Objective: Minimize portfolio risk
obj = cp.Minimize(risk)

# Constraints
constraints = [
    x >= 0, # No short-selling
    x0 >= 0, # No borrowing (long-only)
    R <= ret, # Minimum expected return
    cp.sum(x) + x0 == 1 # Budget constraint (self-financing)
]

# Solve the optimization problem
prob = cp.Problem(obj, constraints)

prob.solve(solver=cp.SCS)
print("Optimal Portfolio Weights:")
print("-----")
for i in range(n):
    print(f"Investment in {tickers[i]}: {round(float(x.value[i]) * 100, 2)}% of portfolio")

# Fix the cash allocation rounding issue
print(f"Cash Allocation: {round(float(x0.value) * 100, 2)}%")
print("-----")

# Fix expected return and risk values
return_percentages = round(float(ret.value) * 100, 2)
risk_value = round(float(risk.value) ** 0.5 * 100, 2)

print(f"Expected Return: {return_percentages}%")
print(f"Expected Risk (Std Dev): {risk_value}%")

```

 Optimal Portfolio Weights:

```

-----
Investment in XLB: -0.0% of portfolio
Investment in XLE: -0.0% of portfolio
Investment in XLF: -0.0% of portfolio
Investment in XLI: -0.0% of portfolio
Investment in XLK: -0.0% of portfolio
Investment in XLP: -0.0% of portfolio
Investment in XLU: -0.0% of portfolio
Investment in XLV: 47.44% of portfolio
Investment in XLY: -0.0% of portfolio
Cash Allocation: 52.56%
-----
Expected Return: 0.08%
Expected Risk (Std Dev): 1.2%
<ipython-input-5-d928e7b60555>:37: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future.
    print(f'Cash Allocation: {round(float(x0.value) * 100, 2)}%')
<ipython-input-5-d928e7b60555>:41: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future.
    return_percentages = round(float(ret.value) * 100, 2)

```

Apply L1 norm

```

mu0 = 0 # Cash holding return (risk-free rate)
n = len(tickers) # Number of assets

# Decision variables
x = cp.Variable(n) # Portfolio weights
x0 = cp.Variable(1) # Cash allocation

# Target return
R = 0.0003

```

```
# Define return and risk expressions
ret = mean_returns.to_numpy() @ x + mu0 * x0 # Portfolio return
risk = cp.quad_form(x, covariance_matrix) # Portfolio risk (quadratic form)

# LASSO Regularization (L1 norm penalty)
alpha = 0.1 # Regularization parameter (tune based on sparsity preference)
lasso_penalty = alpha * cp.norm1(x) # L1 regularization term

# Objective: Minimize portfolio risk with LASSO penalty
obj = cp.Minimize(risk + lasso_penalty)

# Constraints
constraints = [
    x >= 0, # No short-selling
    x0 >= 0, # No borrowing (long-only)
    R <= ret, # Minimum expected return
    cp.sum(x) + x0 == 1 # Budget constraint (self-financing)
]

# Solve the optimization problem
prob = cp.Problem(obj, constraints)

prob.solve(solver=cp.SCS)
print("Optimal Portfolio Weights:")
print("-----")
for i in range(n):
    print(f"Investment in {tickers[i]}: {round(float(x.value[i]) * 100, 2)}% of portfolio")

# Fix the cash allocation rounding issue
print(f"Cash Allocation: {round(float(x0.value) * 100, 2)}%")
print("-----")

# Fix expected return and risk values
return_percentages = round(float(ret.value) * 100, 2)
risk_value = round(float(risk.value) ** 0.5 * 100, 2)

print(f"Expected Return: {return_percentages}%")
print(f"Expected Risk (Std Dev): {risk_value}%")
```

➡ Optimal Portfolio Weights:

```
-----
Investment in XLB: 0.0% of portfolio
Investment in XLE: 0.0% of portfolio
Investment in XLF: 0.0% of portfolio
Investment in XLI: 0.0% of portfolio
Investment in XLK: 0.0% of portfolio
Investment in XLP: 0.0% of portfolio
Investment in XLU: 17.79% of portfolio
Investment in XLV: 0.0% of portfolio
Investment in XLY: 0.0% of portfolio
Cash Allocation: 82.21%
-----
Expected Return: 0.03%
Expected Risk (Std Dev): 0.45%
<ipython-input-14-cd7a952502de>:40: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future.
    print(f"Cash Allocation: {round(float(x0.value) * 100, 2)}%")
<ipython-input-14-cd7a952502de>:44: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future.
    return_percentages = round(float(ret.value) * 100, 2)
```

双击（或按回车键）即可修改

Elastic Net

```
# Target return level
R = 0.0003 # 0.03% daily return

# Risk aversion parameter
lambda_ = 0.5

# Regularization parameters
alpha = 0.01 # L1 regularization (sparsity)
beta = 0.01 # L2 regularization (weight control)

# Objective Function
obj = cp.Maximize(ret - lambda_ * risk - alpha * cp.norm(x, 1) - beta * cp.norm(x, 2)**2)

# Constraints
constraints = [
    x >= 0, # No short-selling
    x0 >= 0, # No borrowing (long-only)
```

```

R <= ret,      # Minimum expected return
cp.sum(x) + x0 == 1  # Budget constraint (self-financing)
]

# Solve the optimization problem
prob = cp.Problem(obj, constraints)
prob.solve(solver=cp.SCS)

# Print results
print("Optimal Portfolio Weights:")
for i in range(n):
    print(f"Investment in {tickers[i]}: {round(float(x.value[i]) * 100, 2)}% of portfolio")

# Fix the cash allocation rounding issue
print(f"Cash Allocation: {round(float(x0.value) * 100, 2)}%")
print("-----")

# Fix expected return and risk values
return_percentages = round(float(ret.value) * 100, 2)
risk_value = round(float(risk.value) ** 0.5 * 100, 2)

print(f"Expected Return: {return_percentages}%")
print(f"Expected Risk (Std Dev): {risk_value}%")

↔ Optimal Portfolio Weights:
Investment in XLB: 0.0% of portfolio
Investment in XLE: -0.0% of portfolio
Investment in XLF: 0.0% of portfolio
Investment in XLI: -0.0% of portfolio
Investment in XLK: 0.0% of portfolio
Investment in XLP: -0.0% of portfolio
Investment in XLU: 17.8% of portfolio
Investment in XLV: -0.0% of portfolio
Investment in XLY: -0.0% of portfolio
Cash Allocation: 82.2%
-----
Expected Return: 0.03%
Expected Risk (Std Dev): 0.45%
<ipython-input-15-8a56396ca86d>:32: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future.
    print(f"Cash Allocation: {round(float(x0.value) * 100, 2)}%")
<ipython-input-15-8a56396ca86d>:36: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future.
    return_percentages = round(float(ret.value) * 100, 2)

```

Trading Constraints

```

mu0 = 0      # Cash holding return (risk-free rate)
n = len(tickers)  # Number of assets

# Decision variables
x = cp.Variable(n)  # Portfolio weights
x0 = cp.Variable(1)  # Cash allocation

original_weights = np.array([0, 0, 0, 0, 0, 0, 0, 0, 0])
wealth = 1000000
trade_limit = 0.05

# Target return
R = 0.0003

# Define return and risk expressions
ret = mean_returns.to_numpy() @ x + mu0 * x0  # Portfolio return
risk = cp.quad_form(x, covariance_matrix)  # Portfolio risk (quadratic form)

# LASSO Regularization (L1 norm penalty)
alpha = 0.1  # Regularization parameter (tune based on sparsity preference)
lasso_penalty = alpha * cp.norm1(x)  # L1 regularization term

# Objective: Minimize portfolio risk with LASSO penalty
obj = cp.Minimize(risk + lasso_penalty)

# Constraints
constraints = [
    x >= 0,      # No short-selling
    x0 >= 0,     # No borrowing (long-only)
    R <= ret,    # Minimum expected return
    cp.sum(x) + x0 == 1,  # Budget constraint (self-financing)
    cp.abs(x - original_weights) * wealth <= trade_limit * volume_data  # Trading constraint
]

# Solve the optimization problem
prob = cp.Problem(obj, constraints)

```

```

prob.solve(solver=cp.SCS)
print("Optimal Portfolio Weights:")
print("-----")
for i in range(n):
    print(f"Investment in {tickers[i]}: {round(float(x.value[i]) * 100, 2)}% of portfolio")

# Fix the cash allocation rounding issue
print(f'Cash Allocation: {round(float(x0.value) * 100, 2)}%')
print("-----")

# Fix expected return and risk values
return_percentages = round(float(ret.value) * 100, 2)
risk_value = round(float(risk.value) ** 0.5 * 100, 2)

print(f"Expected Return: {return_percentages}%")
print(f"Expected Risk (Std Dev): {risk_value}%")

```



ng: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array
 100, 2)}%')

ng: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array
 2)

Trying to Find the Optimal Boundary

✓ Bi-level Portfolio Optimization

We aim to **find the best constraint bounds** ($\theta = \{w_{\min}, w_{\max}, c_{\min}, c_{\max}\}$) that maximize portfolio performance.

Outer Optimization (maximize Sharpe ratio):

$$\max_{\theta \in \Theta} \text{Perf}(w^*(\theta), c^*(\theta)) = \frac{w^T \mu + c \cdot \mu_0}{\sqrt{w^T \Sigma w}}$$

Inner Optimization (portfolio selection):

$$\min_{w, c} w^T \Sigma w + \gamma \cdot \phi_{\text{hold}}(w, c)$$

$$\text{subject to: } w_{\min} \leq w \leq w_{\max}$$

$$c_{\min} \leq c \leq c_{\max}$$

$$\mathbf{1}^T w + c = 1$$

$$w^T \mu + c \cdot \mu_0 \geq R_{\text{target}}$$

$$\|w\|_1 \leq L_{\max}$$

where

$$\phi_{\text{hold}}(w, c) = \kappa_{\text{short}}^T (-w)^+ + \kappa_{\text{borrow}} \cdot (-c)^+$$

```
import itertools
```

```

# Define candidate values to search
x_min_values = [-0.05, -0.02, 0.0]
x_max_values = [0.15, 0.2, 0.25]
c_min_values = [-0.2, -0.1, 0.0]
c_max_values = [0.3, 0.5, 0.7]

# Short-selling allowed up to 5%, 2%, or none
# Max weight per asset
# Cash margin
# Max cash position

# Grid of all combinations

```

```

param_grid = list(itertools.product(x_min_values, x_max_values, c_min_values, c_max_values))

results = []

for x_min_val, x_max_val, c_min_val, c_max_val in param_grid:
    # Apply bounds
    x_min = np.full(n, x_min_val)
    x_max = np.full(n, x_max_val)

    # Define variables and problem as before
    x = cp.Variable(n)
    x0 = cp.Variable()

    ret = mean_returns.to_numpy() @ x + mu0 * x0
    risk = cp.quad_form(x, covariance_matrix)
    lasso_penalty = alpha * cp.norm1(x)

    objective = cp.Minimize(risk + lasso_penalty)

    constraints = [
        x >= x_min, x <= x_max,
        x0 >= c_min_val, x0 <= c_max_val,
        ret >= R,
        cp.sum(x) + x0 == 1
    ]

    prob = cp.Problem(objective, constraints)
    prob.solve(solver=cp.SCS)

    # Skip if infeasible
    if prob.status != 'optimal':
        continue

    # Evaluate performance
    portfolio_return = ret.value
    portfolio_risk = np.sqrt(risk.value)
    sharpe_ratio = portfolio_return / portfolio_risk if portfolio_risk > 0 else 0

    results.append({
        'x_min': x_min_val,
        'x_max': x_max_val,
        'c_min': c_min_val,
        'c_max': c_max_val,
        'return': portfolio_return,
        'risk': portfolio_risk,
        'sharpe': sharpe_ratio
    })

# Convert to DataFrame for easier analysis
import pandas as pd
results_df = pd.DataFrame(results)

# Find best Sharpe ratio
best = results_df.sort_values(by='sharpe', ascending=False).head(1)
print("Best constraint configuration:")
print(best)

```



Best constraint configuration:

	x_min	x_max	c_min	c_max	return	risk	sharpe
68	0.0	0.2	-0.1	0.7	0.0003	0.005766	0.052045

Budget Constraint

✓ Robust Portfolio Optimization with Holding and Trading Costs

We solve:

```

[
    min_w w^T \Sigma w + \alpha \|w\|_1
    subject to: 1^T w + c = 1
                z = w - w^{pre}
                w^{\min} \leq w \leq w^{\max}
                L \leq L^{\text{tar}}
                z^{\min} \leq z \leq z^{\max}
                T \leq T^{\text{tar}}
                c^{\min} \leq c \leq c^{\max}
                \sigma^{\text{wc}} \leq \sigma^{\text{tar}}
]

mu0 = 0 # Cash holding return (risk-free rate)
n = len(tickers) # Number of assets

# Decision variables
x = cp.Variable(n) # Portfolio weights
x0 = cp.Variable(1) # Cash allocation

original_weights = np.array([0, 0, 0, 0, 0, 0, 0, 0])
wealth = 1000000
x_min = np.full(n, 0)
x_max = np.full(n, 0.2)
c_min = -0.1
c_max = 0.7

# Target return
R = 0.0003

# Define return and risk expressions
ret = mean_returns.to_numpy() @ x + mu0 * x0 # Portfolio return
risk = cp.quad_form(x, covariance_matrix) # Portfolio risk (quadratic form)

# LASSO Regularization (L1 norm penalty)
alpha = 0.1 # Regularization parameter (tune based on sparsity preference)
lasso_penalty = alpha * cp.norm1(x) # L1 regularization term

# Objective: Minimize portfolio risk with LASSO penalty
obj = cp.Minimize(risk + lasso_penalty)

# Constraints
constraints = [
    x >= 0, # No short-selling
    x0 >= 0, # No borrowing (long-only cash)
    ret >= R, # Minimum expected return
    cp.sum(x) + x0 == 1, # Budget constraint (self-financing)
    x >= x_min, # Lower bounds
    x <= x_max, # Upper bounds
    x0 >= c_min, # Cash lower bound
    x0 <= c_max
]

# Solve the optimization problem
prob = cp.Problem(obj, constraints)

prob.solve(solver=cp.SCS)
print("Optimal Portfolio Weights:")
print("-----")
for i in range(n):
    print(f"Investment in {tickers[i]}: {round(float(x.value[i]) * 100, 2)}% of portfolio")

# Fix the cash allocation rounding issue
print(f"Cash Allocation: {round(float(x0.value) * 100, 2)}%")
print("-----")

# Fix expected return and risk values
return_percentages = round(float(ret.value) * 100, 2)
risk_value = round(float(risk.value) ** 0.5 * 100, 2)

print(f"Expected Return: {return_percentages}%")
print(f"Expected Risk (Std Dev): {risk_value}%")

➡ Optimal Portfolio Weights:
-----
Investment in XLB: -0.0% of portfolio
Investment in XLE: 0.0% of portfolio
Investment in XLF: 11.21% of portfolio
Investment in XLI: 0.57% of portfolio

```


Investment in XLK: 0.0% of portfolio
 Investment in XLP: 0.0% of portfolio
 Investment in XLU: 12.84% of portfolio
 Investment in XLV: 5.38% of portfolio
 Investment in XLY: -0.0% of portfolio
 Cash Allocation: 70.0%

Expected Return: 0.03%

Expected Risk (Std Dev): 0.58%

<ipython-input-16-93f5703b5637>:51: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future.
 print(f'Cash Allocation: {round(float(x0.value) * 100, 2)}%')

<ipython-input-16-93f5703b5637>:55: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future.
 return_percentages = round(float(ret.value) * 100, 2)

双击（或按回车键）即可修改

3-Months Rolling Window

```
def get_data_for_month_range(df, start_month, end_month, year=2020):
    """
    Filters daily data from df for the given year, from start_month to end_month inclusive.
    Example: get_data_for_month_range(df, 1, 3) -> data for January through March.
    """
    # Boolean mask for rows in [start_month, end_month]
    return df.loc[
        (df.index.year == year) &
        (df.index.month >= start_month) &
        (df.index.month <= end_month)
    ]

def markowitz_optimize(returns_df, target_return=0.8):
    """
    Given daily returns for a set of tickers in returns_df,
    compute mean returns and covariance, then run the Markowitz model
    (long-only, no leverage, target_return constraint) to find optimal weights.
    """
    tickers = returns_df.columns
    mean_returns = returns_df.mean(axis=0).values
    covariance_matrix = returns_df.cov().values
    n = len(tickers)

    # Decision variables
    x = cp.Variable(n, nonneg=True) # weights in each ticker
    x0 = cp.Variable(1, nonneg=True) # cash allocation

    # Portfolio return and risk
    ret = mean_returns @ x
    risk = cp.quad_form(x, covariance_matrix)

    # Objective: minimize risk
    objective = cp.Minimize(risk + lasso_penalty)

    # Constraints
    constraints = [
        x >= 0, # No short-selling
        x0 >= 0, # No borrowing (long-only cash)
        ret >= R, # Minimum expected return
        cp.sum(x) + x0 == 1, # Budget constraint (self-financing)
        x >= x_min, # Lower bounds
        x <= x_max, # Upper bounds
        x0 >= c_min, # Cash lower bound
        x0 <= c_max
    ]

    # Solve
    prob = cp.Problem(objective, constraints)
    prob.solve(solver=cp.SCS) # or another solver if you prefer

    # Extract solutions
    if x.value is None:
        # In case the solver fails, fallback to a naive allocation
        weights = np.zeros(n)
        w_cash = 1.0
    else:
        weights = x.value
        w_cash = x0.value

    return weights, w_cash

def compute_portfolio_performance(test_df, weights, w0):
```

```

"""
Given daily returns for the test month (test_df) and the portfolio weights (weights + w0 for cash),
compute the realized total return over that month.

We'll do a simple approach:
- Each day, the portfolio return = sum(weights * daily_returns).
- Accumulate over the month: (1 + r_1) * (1 + r_2) * ... - 1.
"""
if len(test_df) == 0:
    return 0.0 # no data, no return

daily_returns = test_df.values # shape: [days, n_tickers]
# portfolio return each day (ignoring cash's 0% daily return):
port_ret_daily = np.dot(daily_returns, weights)

# convert daily returns to growth factors:
growth_factors = 1 + port_ret_daily
# total growth over the month:
total_growth = np.prod(growth_factors)
# final total return:
total_return = total_growth - 1.0

return total_return

all_results = []

# We can roll from month 1-3 up to month 9-11
for start_m in range(1, 10): # 1..9
    train_start = start_m
    train_end = start_m + 2
    test_month = start_m + 3

    # 1) Get training data (daily returns) for months [train_start .. train_end]
    train_df = get_data_for_month_range(daily_returns, train_start, train_end)

    # 2) Solve Markowitz on the training data
    w_opt, w_cash = markowitz_optimize(train_df, target_return=0.0003)

    # 3) Test the portfolio on the next month
    test_df = get_data_for_month_range(daily_returns, test_month, test_month)
    realized_return = compute_portfolio_performance(test_df, w_opt, w_cash)

    # 4) Store the results
    result_dict = {
        'Train_Months': f'{train_start}-{train_end}',
        'Test_Month': test_month,
        'Weights': w_opt,
        'Cash': w_cash,
        'Test_Return': realized_return
    }
    all_results.append(result_dict)

# Convert to DataFrame for easier viewing
results_df = pd.DataFrame(all_results)
with pd.option_context('display.max_rows', None, # Show all rows
                        'display.max_columns', None, # Show all columns
                        'display.width', 1000, # Set a wide console width
                        'display.max_colwidth', None): # Don't truncate column text

    print(results_df)

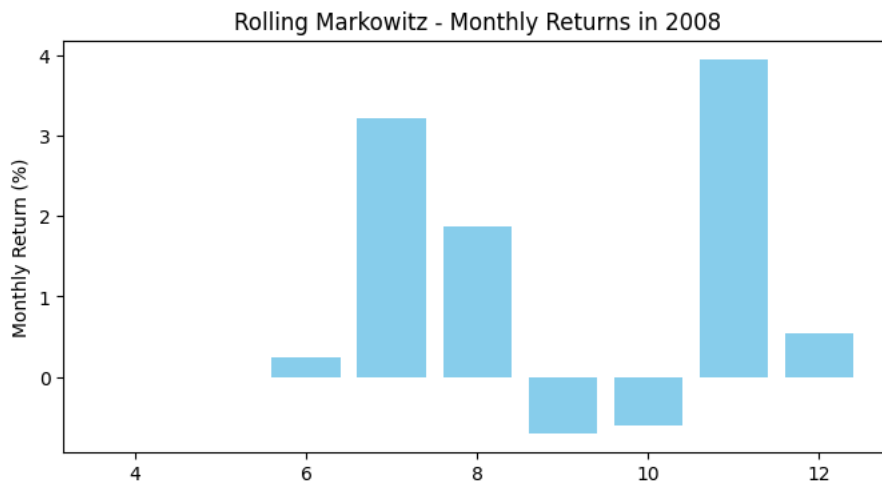

```

	Train_Months	Test_Month	
0	1-3	4	
1	2-4	5	
2	3-5	6	[1.6411201714180354e-06, 3.8391411975550655e-07, 0.012992491410710624, 0.09291582933709701, 3.358247181209042e-06,
3	4-6	7	[1.7981191523568526e-07, 1.3753032379175007e-07, 1.5820005326342717e-07, 0.09999927202478111, 2.501094863034412e-07,
4	5-7	8	[0.0, 0.0101480714,
5	6-8	9	[0.0, 0.033678365,
6	7-9	10	[5.606250188189861e-08, 0.04426085085210811, 0.03220870782173255, 0.08080041299532252,
7	8-10	11	[0.010270258272232904, 0.11099761092624234, 0.05971796491568048, 3.811542384352986e-07, 2.1,
8	9-11	12	[0.003709634875737799, 0.11043884917004876, 1.6053982939024028e-06, 0.0031097681922416776, .

```

plt.figure(figsize=(8, 4))
plt.bar(results_df['Test_Month'], results_df['Test_Return']*100, color='skyblue')
plt.xlabel('Test Month (2008)')
plt.ylabel('Monthly Return (%)')
plt.title('Rolling Markowitz - Monthly Returns in 2008')
plt.show()

```

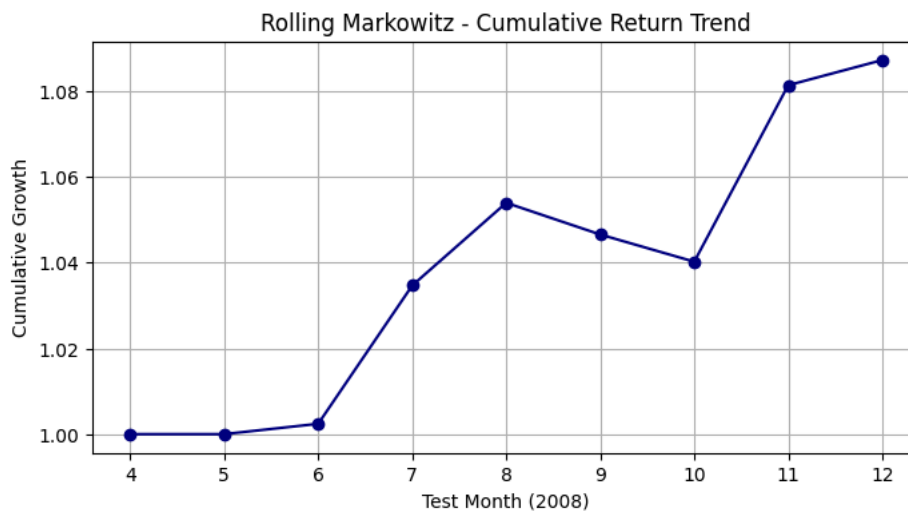


```
# Sort by Test_Month just in case
results_df.sort_values('Test_Month', inplace=True)

# Compute cumulative growth
results_df['Cumulative'] = (1 + results_df['Test_Return']).cumprod()

plt.figure(figsize=(8, 4))
plt.plot(results_df['Test_Month'], results_df['Cumulative'], marker='o', color='navy')
plt.xlabel('Test Month (2008)')
plt.ylabel('Cumulative Growth')
plt.title('Rolling Markowitz - Cumulative Return Trend')
plt.grid(True)
plt.show()

# CVar computation
# experiment with more constraints, more time periods , making records
# markowitz seventy paper
# focusing on risk and return
# good but not too good
# using fresh data is good
```



开始借助 AI 编写或生成代码。